

Light Wand

Design Document - Prototype / Version 1

Date: September 18, 2026

Status: Baseline design established; bench prototype working; power-interface design finalized; full-wand integration next

1. Purpose

The Light Wand is a programmable, battery-powered LED wand intended primarily for long-exposure photography and generative light painting.

The core idea is to treat the physical wand as a moving display surface. A computer can generate a changing one-dimensional slice of pixels, stream those colors to the wand, and allow the wand's physical motion through space to turn those slices into a two-dimensional photographic image.

The system is deliberately modular. The first version should prove the light-output, power, physical, and wireless architecture without making the wand mechanically complicated. Motion sensing and more advanced interaction can be layered in later.

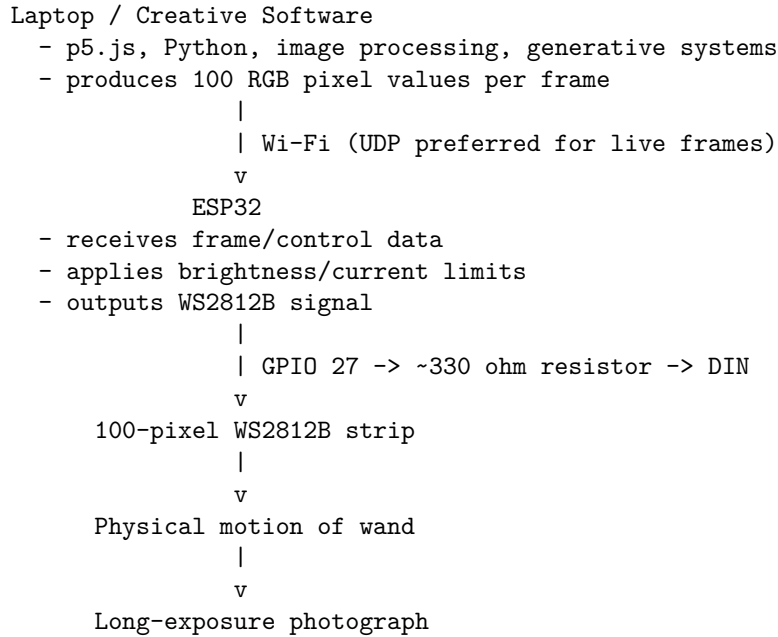
2. Design Goals

The baseline design should:

- Drive **100 individually addressable WS2812B LEDs** from an ESP32.
 - Remain **portable and untethered** during photography.
 - Support both **self-contained LED patterns** and **real-time pixel streaming from a laptop**.
 - Allow firmware updates over **Wi-Fi OTA** after the initial USB upload.
 - Preserve mounting options for handheld, pivoted, pendulum, tripod/clamp, and constrained-motion experiments.
 - Keep the physical build simple enough to modify as experiments reveal what matters.
 - Leave a clear upgrade path for an **IMU** so motion can eventually influence or synchronize generated imagery.
-

3. System Architecture

3.1 High-Level Data Flow



This separates **image/pattern generation** from **real-time LED driving**. The laptop can do computationally expensive or highly experimental work while the ESP32 remains a lightweight real-time receiver and LED controller.

4. Core Hardware

Component	Baseline choice	Notes
Microcontroller	ESP32 development board	Wi-Fi, OTA capability, sufficient processing and memory
LED strip	WS2812B, 5 V, 100 individually addressable LEDs	Current strip is 3 m total; project uses 100 pixels
LED data pin	GPIO 27	Current physical wiring baseline
Data resistor	~330 ohms	In series between GPIO 27 and LED DIN, preferably near ESP32
Power source	INIU P55 USB power bank	Native 5 V / 3 A USB-C output used for v1

Component	Baseline choice	Notes
USB-C power interface	Teansic USB-C sink/female breakout	5.1 k Ω CC pull-downs establish a proper USB-C sink and expose regulated 5 V VBUS
Physical support	Black-painted wood strip	Lightweight, rigid, easy to drill and modify
Wiring	Flexible stranded wire	Data lead runs alongside strip to DIN; power wiring kept short where practical
Fastening	LED adhesive + small hot-glue retention dots	Hot glue used sparingly as mechanical retention

4.1 Future Hardware

A **BNO085/BNO086-class IMU** is the preferred future motion-sensing option. It provides fused orientation/quaternion data and avoids much of the drift-management work required when using a basic accelerometer/gyro alone.

The IMU is intentionally deferred until the full LED streaming architecture is reliable.

5. Electrical Architecture

5.1 LED Signal Wiring

Current baseline:

```

ESP32 GPIO 27 ---- 330 ohm resistor -----> WS2812B DIN
ESP32 GND -----> Common GND
5 V supply -----> LED +5 V
Supply GND -----> LED GND

```

The ESP32 and LED strip must share a common ground.

Because the strip's **DIN is at the short end of the physical wand**, while the electronics are planned at the opposite end, the data conductor runs alongside the LED strip back to DIN. Small hot-glue dots may be used to secure that wire along the wand.

The arrows printed on the WS2812B strip determine data direction. The ESP32 must feed the **DIN/input side**, not the output side.

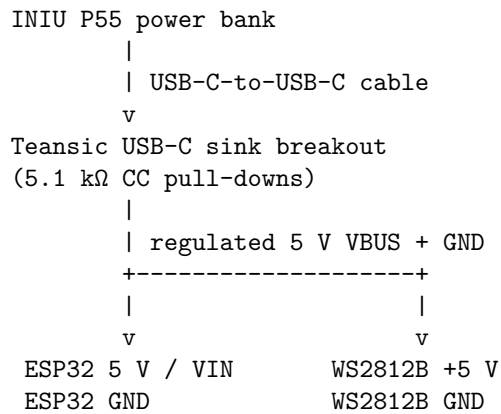
5.2 Power Distribution and USB-C Interface

The finalized v1 power architecture uses the INIU P55 in its native **5 V / 3 A** mode. A **USB-C-to-USB-C cable** connects the power bank to a **Teansic USB-C sink/female breakout board** containing the required **5.1 kΩ CC pull-down resistors**. Those resistors identify the breakout as a USB-C power sink so the power bank presents standard regulated 5 V on VBUS.

This breakout board is therefore part of the baseline design. It is not being used to boost or reduce voltage; its job is to provide the correct USB-C sink interface and expose a safe, stable 5 V/GND connection for the wand.

The previously tested exposed-wire USB-C pigtail is **not suitable for this role**. It was intended for the opposite power-flow direction, and testing at its bare leads showed an unstable reading of approximately **8-11 V** instead of the expected fixed 5 V. It should not be connected to the ESP32 or LED strip.

The complete v1 power path is:



The full-strip LED current should **not** be routed through an ESP32 board trace or regulator. Power branches in parallel after the USB-C sink breakout, while the ESP32 and LED strip share common ground for reliable data signaling.

A USB-PD trigger board plus separate 5 V buck converter were considered but are **not part of v1**. The simpler native-5-V architecture was selected to reduce size, heat, wiring, and complexity. A higher-power PD/buck arrangement remains only a possible future upgrade if experiments show that substantially more current is needed.

5.3 Brightness / Current Assumption

The INIU P55 can provide a **5 V / 3 A** profile. The software should still enforce a conservative brightness/current ceiling rather than assuming every pixel can run at full white simultaneously.

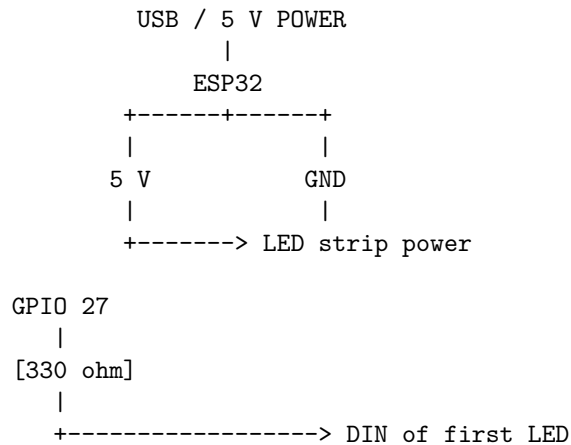
For early full-strip testing:

- Begin at low global brightness.
- Test progressively: **10 -> 25 -> 50 -> 100 LEDs**.
- Use a software current limit of roughly **2.5 A** initially, leaving margin below the nominal 3 A source capability.
- Avoid designing patterns around sustained full-brightness white across all 100 LEDs.

The light-painting use case generally does not require maximum electrical output; exposure time, camera settings, motion speed, and LED brightness all contribute to the photographed result.

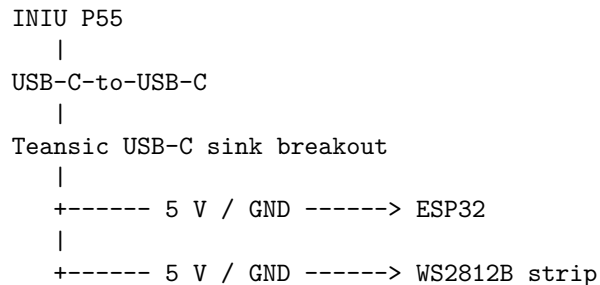
6. Current 10-LED Bench Configuration

The small test configuration is intentionally simple and validates the essential signal chain before scaling to the full wand.



LED count in firmware: 10

For the 100-pixel wand, the power architecture changes so the full LED current does not pass through the ESP32 board:



The signal path remains **GPIO 27 -> 330 ohm resistor -> DIN**, with a common ground shared by the ESP32 and LED strip.

7. Physical Wand Construction

7.1 Layout

The current mechanical concept is:

```
[ DIN / LED 1 ]===== [ LED 100 ] [ ELECTRONICS ]
      ^                                     ~8 in reserved area
      |
      +--- data wire runs along the side of the strip ----- GPIO 27
```

Key assumptions:

- The LED strip remains attached to a straight wood support.
- The wood is painted **black** to minimize visibility in long exposures.
- Approximately **8 inches** at one end are reserved for the ESP32, power connections, switches/controls, and enclosure development.
- The LED strip is held by its factory adhesive, supplemented with **small spaced hot-glue dots** where needed.
- The long data wire runs along the side of the strip and is also retained with small glue dots.
- Existing unused strip connectors may remain available during prototyping unless they interfere mechanically.

7.2 Mounting Provision

The wand should preserve mounting points rather than optimize only for hand-held use.

Desired experimental configurations include:

- Handheld sweeping and drawing
- End pivot
- Center pivot
- Pendulum suspension
- Tripod or clamp mounting
- Repeatable constrained-motion rigs

A simple drilled hole or eye-screw mounting provision near an end can support pendulum experiments. Center/back mounting space should remain available for future fixtures.

7.3 Weight Distribution

Battery and electronics placement affects pendulum behavior. For the first build, they should remain mechanically accessible rather than permanently optimized for one motion mode. Different experiments may benefit from moving mass toward the pivot, center, or end.

8. Firmware Architecture

The ESP32 firmware should be divided into small independent responsibilities:

1. **LED output** - FastLED or equivalent drives the WS2812B strip.
2. **Pattern engine** - optional onboard patterns for testing and untethered standalone use.
3. **Wireless receiver** - receives pixel frames or compact control messages.
4. **Brightness/current limiter** - prevents unsafe or unnecessary power draw.
5. **OTA update service** - allows firmware updates over Wi-Fi after initial setup.
6. **Future controls** - buttons, knob/encoder, mode selection, etc.
7. **Future IMU service** - orientation and motion data without coupling it directly to the LED driver.

Keeping these concerns separate is important because the experimental direction of the project is expected to evolve.

9. Wireless Design

9.1 Two Different Wi-Fi Functions

Wi-Fi serves two separate purposes:

OTA firmware updates

Used to upload revised ESP32 code without plugging the wand into USB every time.

Real-time control/data streaming

Used while the wand is running to transmit colors, patterns, parameters, or pixel frames from a laptop.

These should remain conceptually and architecturally separate even if they share the same Wi-Fi connection.

9.2 Real-Time Pixel Streaming

A 100-pixel RGB frame contains:

100 pixels x 3 bytes/pixel = 300 bytes/frame

At 60 frames/second:

300 bytes x 60 = 18,000 bytes/sec

That data rate is trivial for Wi-Fi.

UDP is the preferred transport for live pixel frames because latency matters more than guaranteed delivery. If an occasional frame is lost, the next complete frame simply replaces it.

A simple frame protocol can begin with raw 300-byte RGB packets. Headers, sequence numbers, checksums, or mode identifiers can be added later if experiments show they are useful.

9.3 Startup / Synchronization

The laptop does not need to begin at the exact instant the ESP32 powers on.

Normal behavior can be:

1. ESP32 boots.
2. ESP32 connects to Wi-Fi.
3. ESP32 opens its UDP listener.
4. Laptop continuously transmits complete frames.
5. ESP32 starts displaying whichever valid frame arrives next.

For experiments requiring a repeatable synchronized start, a lightweight **READY / START handshake** can be added later.

9.4 Bluetooth

Bluetooth remains technically possible but is **not part of the current baseline architecture**. Wi-Fi is the better fit for full-frame streaming and OTA firmware updates. Bluetooth may still make sense later for simple control commands, but it is not required for version 1.

10. Computer-Side Creative Pipeline

The laptop is the creative engine. Candidate sources include:

- p5.js generative sketches
- Python generative systems
- Still-image sampling
- Procedural animation
- Music/audio analysis
- Video or camera-derived data
- Existing code-art systems

A central concept is to generate a sequence of **100-color vertical slices**. Each slice becomes one instantaneous state of the wand. The motion of the wand across the scene supplies the missing spatial dimension in the long exposure.

This makes the wand less like a conventional light-painting prop and more like a **physical raster display whose second axis is created by movement**.

11. Motion Sensing - Future Phase

Once streaming is stable, a BNO085/BNO086-class IMU can be added via I2C.

Typical connections:

```
ESP32 3.3 V ---> IMU VIN / 3V3
ESP32 GND  ---> IMU GND
ESP32 SDA  ---> IMU SDA
ESP32 SCL  ---> IMU SCL
```

Potential uses include:

- Detecting direction changes
- Advancing frames based on actual wand movement rather than elapsed time
- Varying color, density, or pattern behavior with rotation
- Measuring orientation for repeatable sweeps
- Using motion as a generative input
- Making the computer-generated image respond to the physical performer

The important design principle is that motion sensing should **augment**, not block, the simpler streaming system.

12. Controls - Future Phase

Local controls may eventually be useful even when the laptop is not present.

Possibilities include:

- Power switch
- Brightness control
- Pattern/mode selector
- Speed control
- Direction/reverse control
- Start/stop or trigger button

These controls should send parameters into the same firmware architecture rather than each creating a separate hard-coded behavior.

A later phone/web interface could expose similar controls over Wi-Fi.

13. Software Safety Rules

Firmware should enforce several conservative limits:

- Define the actual LED count explicitly.
 - Apply a global brightness limit.
 - Apply a software current limit.
 - Default to LEDs off or a low-power state after boot until initialization succeeds.
 - Validate incoming packet sizes before copying RGB data.
 - Ignore malformed wireless packets rather than allowing them to corrupt the LED buffer.
 - Keep OTA update mode from accidentally driving a high-power pattern indefinitely.
-

14. Development Sequence

The recommended sequence is intentionally incremental:

Phase 1 - Bench proof

- ESP32 communicates with a short LED section.
- GPIO 27 + resistor + DIN direction verified.
- Basic FastLED/NeoPixel effects work.

Status: working.

Phase 2 - Full strip and power

- Install and verify the Teensic USB-C sink breakout using a USB-C-to-USB-C cable from the INIU P55.
- Confirm stable regulated 5 V at the breakout before connecting the full wand.
- Establish parallel 5 V power branching to the ESP32 and LED strip.
- Start at low brightness.
- Progressively test 10, 25, 50, then 100 LEDs.
- Confirm stable color/data operation.

Phase 3 - Physical wand

- Attach strip to black support.
- Secure long data wire.
- Mount ESP32/power hardware in the reserved electronics area.
- Preserve access for modification and mounting experiments.

Phase 4 - Wi-Fi streaming

- ESP32 connects to local Wi-Fi.
- Laptop transmits 300-byte RGB frames via UDP.
- ESP32 displays the newest complete frame immediately.
- Test p5.js/Python-generated slice sequences.

Phase 5 - OTA updates

- Enable reliable OTA firmware upload.
- USB becomes primarily a recovery/debug path rather than the normal programming workflow.

Phase 6 - Motion experiments

- Add BNO085/BNO086-class IMU.
- Explore movement-triggered frame advancement and closed-loop generative behaviors.

15. Key Assumptions and Decisions

The following are considered the current baseline rather than open questions:

- **ESP32** is the wand controller.
- The output device is a **100-pixel WS2812B 5 V strip**.
- **GPIO 27** is the current LED data output.
- A **~330 ohm series resistor** is used on the data line.
- The LED strip and ESP32 use a **common ground**.
- Full LED current should not be routed through the ESP32 board.
- The INIU P55's **5 V / 3 A** output is sufficient for conservative prototype operation with software current limiting.
- The v1 USB-C power interface is the **Teansic USB-C sink/female breakout with 5.1 kΩ CC pull-downs**, connected by a USB-C-to-USB-C cable.
- The previously tested exposed-wire USB-C pigtail is **not used** because it is the wrong-direction cable for this application and produced an unstable approximately **8-11 V** reading.
- A USB-PD trigger and separate 5 V buck converter are **not part of v1**.
- The wand uses a **black wood support** for the first physical build.
- DIN is at the far/short end, so a **long data wire runs alongside the strip**.
- Small **hot-glue dots** are acceptable for wire and strip retention.
- The primary live-data transport is **Wi-Fi**, with **UDP** favored for RGB frames.
- **OTA over Wi-Fi** is desirable for future firmware updates.
- **Bluetooth is not part of the v1 baseline**.

- Motion sensing is deferred; **BNO085/BNO086** is the preferred direction when added.
 - The system should remain modular and experimentally reconfigurable rather than optimized too early around one photographic motion.
-

16. Open Design Questions

These are intentionally left open until real use provides evidence:

- Final electronics enclosure shape and attachment method
 - Whether controls belong directly on the wand and which controls earn permanent hardware
 - Exact full-strip brightness/current ceiling after testing
 - Final cable/connectors for a more durable version
 - Whether power injection is necessary at more than one point for the intended brightness levels
 - Whether laptop streaming should use raw RGB only or a small framed protocol
 - Whether an ESP32-created access point is preferable to joining an existing Wi-Fi network for field use
 - Best IMU mounting location and calibration strategy
 - Which motion constraints create the most compelling photographic results
-

17. Design Philosophy

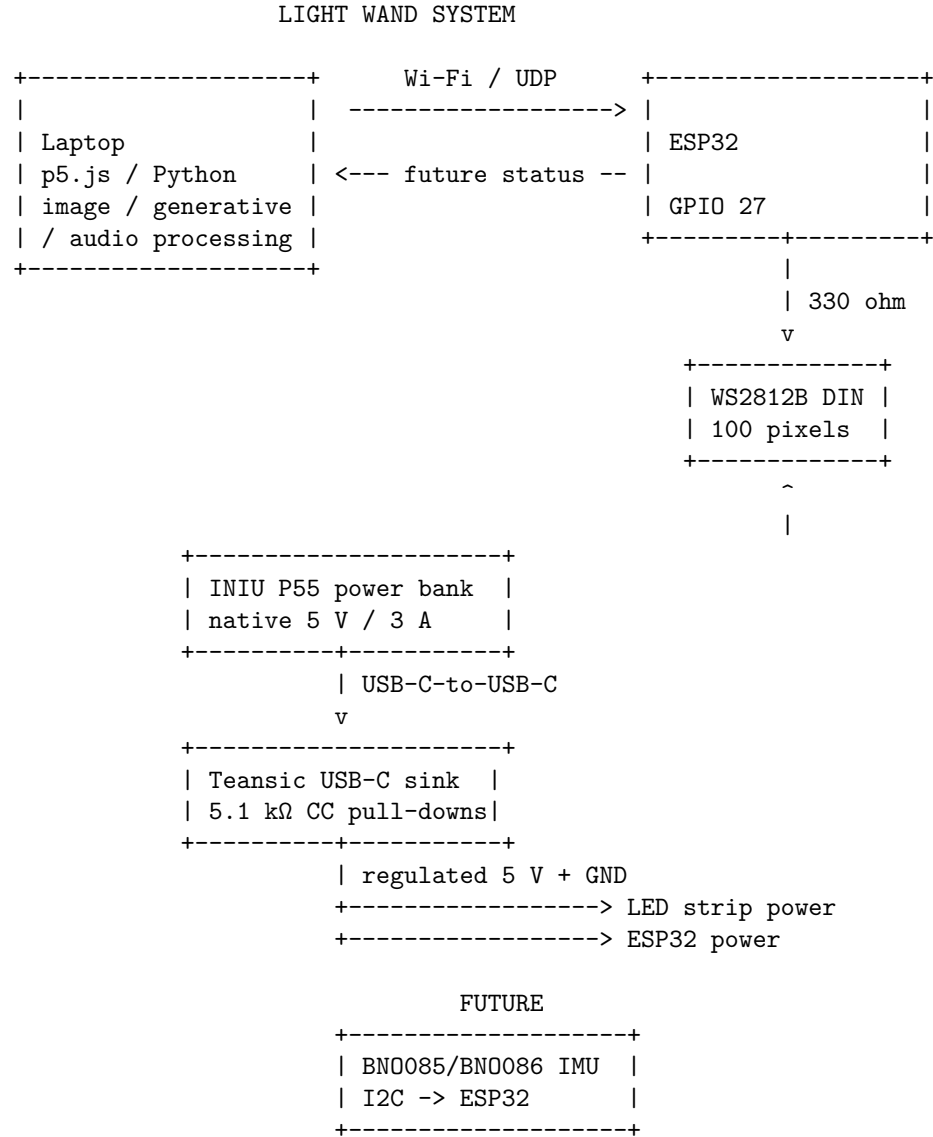
The Light Wand is not being designed as a finished commercial object yet. It is an **experimental instrument**.

The first version should make it easy to ask questions such as:

- What happens when the LED image is generated continuously instead of preloaded?
- What kinds of physical motion produce recognizable or surprising spatial structures?
- Can motion become part of the generative feedback loop?
- What happens when algorithmic imagery is translated into a physical gesture and then reconstructed photographically?

The hardware is therefore intentionally straightforward. The interesting part of the project is the interaction between **code, light, motion, time, and photography**.

18. Baseline Block Diagram



Document intent: This is the working baseline for the Light Wand prototype. Changes should be driven by what is learned from building and photographing with the system, not by adding complexity in advance.